



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 06 - VECTOR DATABASE



OBJETIVOS DA AULA

- ❑ Aprofundar o conceito de Embeddings como a base da busca semântica.
- ❑ Entender as limitações arquiteturais do InMemoryEmbeddingStore para ambientes de produção.
- ❑ Justificar a necessidade de um Vector Database dedicado para persistência, escalabilidade e funcionalidades avançadas.
- ❑ Apresentar o ecossistema de integrações de EmbeddingStore do Langchain4j, comparando as principais opções.



O QUE SÃO EMBEDDINGS?



O QUE SÃO EMBEDDINGS?

- ❑ Como vimos, um LLM processa números, não texto. O processo de RAG depende fundamentalmente de Embeddings para funcionar.
- ❑ Um embedding é uma representação vetorial (uma lista de números de ponto flutuante) de um trecho de texto em um espaço de alta dimensão. Este vetor captura o significado semântico do texto.
- ❑ Como são Criados?
 - ❑ Um EmbeddingModel é um modelo treinado para essa tarefa.
 - ❑ Ele analisa o texto e o posiciona como um ponto em um "mapa de significados" multidimensional.
 - ❑ Textos com significados semelhantes, como "Qual a política de cancelamento?" e "Como faço para cancelar minha reserva?", serão mapeados para pontos muito próximos nesse espaço.
- ❑ A Busca Semântica
 - ❑ Quando o usuário faz uma pergunta, o embedding da pergunta é gerado e é feita uma busca no banco de dados dos embeddings de documentos que estão mais "próximos" no mapa, usando métricas como a similaridade de cossenos.

O PAPEL DA VECTOR API (JEP 508)

O PAPEL DA VECTOR API (JEP 508)

- ❑ Otimização de Baixo Nível
 - ❑ A busca por similaridade envolve cálculos matemáticos intensivos entre vetores. O Java está evoluindo para acelerar essas operações diretamente no hardware.
- ❑ O que é a Vector API?
 - ❑ Iniciada com o JEP 338 e agora em sua décima incubação com o JEP 508 (para o JDK 25), a Vector API expõe as instruções SIMD (Single Instruction, Multiple Data) da CPU para o código Java.
- ❑ Como funciona?
 - ❑ Em vez de somar dois números de cada vez em um loop (operação escalar), a Vector API permite que o Java instrua a CPU a somar, por exemplo, 8 pares de números de uma só vez (operação vetorial).
- ❑ Relevância para IA
 - ❑ Embora frameworks como o Langchain4j abstraíam isso de nós, a maturação da Vector API no JDK significa que as bibliotecas de baixo nível e os próprios Vector Databases poderão executar buscas de similaridade de forma muito mais performática no futuro, abrindo ainda mais possibilidades para o uso de Java no mundo de IA.

VECTOR DATABASE

VECTOR DATABASE

- ❑ Até então, usamos o InMemoryEmbeddingStore, que o EasyRAG nos forneceu. Ele é fantástico para casos mais simples, mas apresenta duas limitações críticas para aplicação de produção :
 - ❑ Volatilidade
 - ❑ Como o nome sugere, ele armazena todos os embeddings na memória RAM. Se a aplicação for reiniciada, toda a base de conhecimento é perdida. O custoso processo de ingestão (Load, Split, Embed) precisa ser executado novamente a cada deploy, o que pode se tornar inviável.
 - ❑ Escalabilidade Limitada
 - ❑ O tamanho da base de conhecimento está diretamente limitado pela quantidade de memória RAM disponível em uma única instância da aplicação.
 - ❑ Fica muito difícil escalar para grandes volumes de documentos ou distribuir a carga entre múltiplos servidores.

VECTOR DATABASE

- ❑ O que é um Vector Database?
 - ❑ É um banco de dados especializado, projetado desde o início para armazenar, indexar e consultar eficientemente um grande número de embeddings de alta dimensão.
- ❑ Por que usá-lo?
 - ❑ Persistência: Os embeddings são armazenados em disco, sobrevivendo a reinicializações da aplicação. A ingestão de dados é feita uma vez (ou incrementalmente), não a cada startup.
 - ❑ Escalabilidade: Projetados para escalar horizontalmente, distribuindo bilhões de vetores em clusters de servidores.
 - ❑ Performance: Utilizam algoritmos de busca otimizados (como HNSW - Hierarchical Navigable Small World) para encontrar os vizinhos mais próximos aproximados (ANN) em milissegundos, mesmo em datasets massivos.
 - ❑ Funcionalidades Avançadas: Oferecem recursos essenciais para produção, como filtragem por metadados, atualizações em tempo real e APIs de gerenciamento robustas.

VECTOR DATABASE VS. VECTOR STORE

VECTOR DATABASE VS. VECTOR STORE

- ❑ Embora às vezes usados de forma intercambiável, esses termos representam duas camadas diferentes da nossa arquitetura.
 - ❑ Vector Database: Refere-se à tecnologia subjacente, o sistema de banco de dados real que armazena, gerencia e indexa os vetores. Exemplos: Qdrant, PgVector, Milvus. É o "motor".
 - ❑ Vector Store (ou EmbeddingStore no Langchain4j): Refere-se à abstração no código da nossa aplicação. É a interface ou o cliente que usamos para interagir com o Vector Database.
- ❑ Analogia com JDBC:
 - ❑ O Vector Database é como o PostgreSQL ou o Oracle Database em si.
 - ❑ O Vector Store (EmbeddingStore) é como a interface `javax.sql.DataSource` ou o objeto `Connection` no nosso código Java. É através dele que enviamos nossas "consultas" (buscas por similaridade).
- ❑ Essa distinção é fundamental para entender o poder do Langchain4j
 - ❑ Nossa aplicação depende da interface `EmbeddingStore`, não de uma implementação concreta.
 - ❑ Isso nos permite trocar o Vector Database (de PgVector para Qdrant, por exemplo) alterando apenas a configuração e a dependência, sem tocar na lógica de negócio.



VECTOR STORES COM LANGCHAIN4J

- ❑ Uma das maiores vantagens do Langchain4j é sua arquitetura de EmbeddingStore baseada em interfaces.
 - ❑ Isso nos permite trocar o banco de dados vetorial com pouca ou nenhuma alteração no código da nossa aplicação, apenas mudando a dependência e a configuração.
- ❑ O Langchain4j suporta uma vasta gama de integrações, incluindo :
 - ❑ Bancos de Dados Vetoriais Dedicados: Qdrant, Milvus, Weaviate, Pinecone.
 - ❑ Extensões para Bancos de Dados Existentes: PgVector (para PostgreSQL), Neo4j, Redis, MongoDB Atlas Vector Search.
 - ❑ Soluções em Nuvem e Locais.

RESUMO E PRÓXIMOS PASSOS

RESUMO E PRÓXIMOS PASSOS

- ❑ O que Aprendemos?
 - ❑ Embeddings são a base da busca semântica, e a Vector API do Java promete acelerar os cálculos múltiplos no nível dos registradores.
 - ❑ O InMemoryEmbeddingStore é inadequado para produção devido à sua volatilidade e falta de escalabilidade.
 - ❑ Um Vector Database é a solução de nível de produção para armazenar embeddings de forma persistente, escalável e performática.
 - ❑ O Vector Store (EmbeddingStore) é a interface em nosso código que abstrai a comunicação com o Vector Database.
 - ❑ O Langchain4j oferece um ecossistema rico de integrações, permitindo-nos escolher o Vector DB que melhor se adapta às nossas necessidades de projeto.
- ❑ Próxima Aula:
 - ❑ Vamos colocar a mão na massa. Escolheremos um Vector Database (o PgVector), configurá-lo, remover a dependência do easy-rag e implementar nosso próprio pipeline de ingestão de dados programático e persistente.

ATÉ A PRÓXIMA!